

Concert Booking - MEAN Stack

Reference Solution and Explanation

What this question is testing

- Fetching list data from the backend and rendering it in Angular.
- Searching/filtering records in the frontend.
- Validating user input before calling the backend.
- Handling sold-out and over-request business rules correctly.
- Calling a PATCH endpoint with the exact request body shape.
- Returning exact status codes and success messages expected by hidden tests.

Backend solution

Model: src/models/concert.js

```
const mongoose = require('mongoose');

const concertSchema = new mongoose.Schema({
  concert_name: { type: String, required: true },
  date: { type: Date, required: true },
  location: { type: String, required: true },
  available_tickets: { type: Number, required: true }
}, { versionKey: false });

module.exports = mongoose.model('concert', concertSchema);
```

Router: src/routers/concert.js

```
const express = require('express');
const router = express.Router();
const Concert = require('../models/concert');

router.get('/concert', async (req, res) => {
  try {
    const concerts = await Concert.find();
    return res.status(200).json(concerts);
  } catch (error) {
    return res.status(400).json({ message: 'Unable to fetch concerts' });
  }
});

router.patch('/concert/:id', async (req, res) => {
  try {
    const { id } = req.params;
    const { tickets } = req.body;

    if (tickets === undefined || tickets === null || tickets === '' ||
        isNaN(tickets)) {
      return res.status(400).json({ message: 'Invalid ticket count' });
    }
  }
});
```

```

const ticketCount = Number(tickets);
if (ticketCount <= 0) {
  return res.status(400).json({ message: 'Invalid ticket count' });
}

const concert = await Concert.findById(id);
if (!concert) {
  return res.status(400).json({ message: 'Concert not found' });
}

if (ticketCount > concert.available_tickets) {
  return res.status(400).json({ message: 'Not enough tickets available' });
}

concert.available_tickets = concert.available_tickets - ticketCount;
await concert.save();

return res.status(200).json({ message: 'Ticket booked successfully' });
} catch (error) {
  return res.status(400).json({ message: 'Unable to update tickets' });
}
});

module.exports = router;

```

App wiring

```

// src/app.js
const express = require('express');
const cors = require('cors');
const concertRoutes = require('./routers/concert');
const app = express();
app.use(cors());
app.use(express.json());
app.use('/', concertRoutes);
module.exports = app;

// src/server.js
const mongoose = require('mongoose');
const app = require('./app');
mongoose.connect('mongodb://127.0.0.1:27017/concert')
  .then(() => app.listen(3000))
  .catch(console.error);

```

Why the backend is written this way

- The field names must remain exactly concert_name, date, location, and available_tickets because Fresco-style tests often depend on exact names.
- The GET route returns status 200 on success and 400 on failure because that matches the screenshot instructions.
- The PATCH route uses the exact body key tickets, validates it, checks stock, decrements available_tickets, and returns the exact success message Ticket booked successfully.
- Validation is done before database update so invalid inputs never modify the collection.

Angular solution (standalone style)

Service: src/app/concert.service.ts

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class ConcertService {
  private baseUrl = 'http://localhost:3000/concert';
  constructor(private http: HttpClient) {}
  getConcerts(): Observable<any> { return this.http.get(this.baseUrl); }
  bookTickets(id: string, tickets: number): Observable<any> {
    return this.http.patch(`${this.baseUrl}/${id}`, { tickets });
  }
}

```

Bootstrap/config

```

// src/app/app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient } from '@angular/common/http';
export const appConfig: ApplicationConfig = { providers: [provideHttpClient()] };

// src/main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app';
bootstrapApplication(AppComponent, appConfig).catch((err: unknown) =>
  console.error(err));

```

Component logic: src/app/app.ts

```

import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { ConcertService } from '../concert.service';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class AppComponent implements OnInit {
  concerts: any[] = [];
  filteredConcerts: any[] = [];
  constructor(private concertService: ConcertService) {}

  ngOnInit(): void { this.loadConcerts(); }

  loadConcerts(): void {
    this.concertService.getConcerts().subscribe({
      next: (response: any) => { this.concerts = response; this.filteredConcerts = response; },
      error: (err: any) => console.error(err)
    });
  }

  searchConcert(event: Event): void {
    const value = (event.target as HTMLInputElement).value.toLowerCase().trim();
    this.filteredConcerts = this.concerts.filter((concert) =>
      concert.concert_name.toLowerCase().includes(value)
    );
  }
}

```

```

    );
  }

  bookConcert(concert: any): void {
    if (concert.available_tickets === 0) { alert('The tickets are sold out');
return; }
    const inputValue = prompt('Enter the ticket count');
    if (!inputValue || !/^[0-9]+$/.test(inputValue.trim())) { alert('Input field
should be valid'); return; }
    const ticketCount = Number(inputValue);
    if (ticketCount <= 0) { alert('Input field should be valid'); return; }
    if (ticketCount > concert.available_tickets) {
      alert(`Sorry, we only have ${concert.available_tickets} tickets available.
Your request for ${ticketCount} tickets cannot be fulfilled.`);
      return;
    }
    this.concertService.bookTickets(concert._id, ticketCount).subscribe({
      next: () => { alert('Tickets booked successfully'); this.loadConcerts(); },
      error: (err: any) => console.error(err)
    });
  }
}

```

Template and CSS

```

<!-- src/app/app.html -->
<div class="page">
  <div class="header">
    <h1>Book My Concert</h1>
    <p>Concert Booking - MEAN Stack</p>
  </div>
  <div class="search-box">
    <input type="text" placeholder="Enter the concert name here"
(input)="searchConcert($event)" />
  </div>
  <div class="concert-grid">
    <div class="concert-card" *ngFor="let concert of filteredConcerts">
      <h2>{{ concert.concert_name }}</h2>
      <p><strong>Date:</strong> {{ concert.date | date:'dd/MM/yyyy' }}</p>
      <p><strong>Location:</strong> {{ concert.location }}</p>
      <p><strong>Available Tickets:</strong> {{ concert.available_tickets }}</p>
      <button (click)="bookConcert(concert)">Book</button>
    </div>
  </div>
</div>

/* src/app/app.css */
.page { min-height: 100vh; background: linear-gradient(135deg, #eef2ff, #f8fafc);
padding: 24px; font-family: Arial, sans-serif; }
.header { text-align: center; margin-bottom: 24px; }
.search-box { max-width: 500px; margin: 0 auto 24px auto; }
.search-box input { width: 100%; padding: 12px 14px; border: 1px solid #cbd5e1;
border-radius: 12px; }
.concert-grid { max-width: 1200px; margin: 0 auto; display: grid; grid-template-
columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 20px; }
.concert-card { background: #fff; border-radius: 18px; padding: 20px; box-shadow: 0
10px 25px rgba(15,23,42,0.08); border: 1px solid #e2e8f0; }
.concert-card button { margin-top: 12px; padding: 10px 14px; border: none; border-

```

```
radius: 10px; background: #2563eb; color: white; font-weight: 600; cursor: pointer;
}
```

Why the frontend is written this way

- The concert list is loaded on init and copied into both `concerts` and `filteredConcerts` so search can work without losing the original data.
- The search uses `concert_name.toLowerCase().includes(value)`, which is simple and usually test-friendly for partial matching.
- The booking function validates in a strict order: sold out first, invalid input next, non-positive values next, then over-request check, and only then calls the backend.
- The frontend alert on success is `Tickets booked successfully`, while the backend response is `Ticket booked successfully`. Keeping those different strings correct is important for hidden tests.
- After a successful booking, the list is reloaded so the updated ticket count is shown immediately.

Likely hidden test cases

- `GET /concert` returns status 200 and the complete concert list.
- Searching with partial input shows only matching concert cards.
- Entering letters, special characters, or blank spaces shows Input field should be valid.
- Clicking Book when `available_tickets` is 0 shows The tickets are sold out.
- Entering a ticket count greater than `available_tickets` shows the exact over-request message.
- The PATCH body must be `{ "tickets": <number> }`.
- PATCH success returns status 200 with `Ticket booked successfully`.
- The frontend shows `Tickets booked successfully` and refreshes the list after a successful booking.